



ROYAUME DU MAROC

,Ministère de l'Enseignement Supérieur,
de la Formation des Cadres et de la Recherche Scientifique

CONCOURS NATIONAL COMMUN
d'Admission dans les Établissements de Formation
d'Ingénieurs et Établissements Assimilés
Édition 2021 - 2015

ÉPREUVE D'INFORMATIQUE

Partie : Programmation python

Préparation CNC : Informatique

Filières : **MP/PSI/TSI**

Durée **2 heures**

Les candidats sont informés que la précision des raisonnements algorithmiques ainsi que le soin apporté à la rédaction et à la présentation des copies seront des éléments pris en compte dans la notation. Il convient en particulier de rappeler avec précision les références des questions abordées. Si, au cours de l'épreuve, un candidat repère ce qui peut lui sembler être une erreur d'énoncé, il le signale sur sa copie et poursuit sa composition en expliquant les raisons des initiatives qu'il est amené à prendre.

Remarques générales :

- ❖ Cette épreuve est composée d'un exercice et de trois parties tous indépendants ;
- ❖ Toutes les instructions et les fonctions demandées seront écrites en Python ;
- ❖ Les questions non traitées peuvent être admises pour aborder les questions ultérieures ;
- ❖ Toute fonction peut être décomposée, si nécessaire, en plusieurs fonctions.

~~~~~

Important : Le candidat doit impérativement traiter l'exercice ci-dessous, et inclure les réponses dans les premières pages du cahier de réponse.

Exercice : (4 points)

*Développement limité du cosinus*

Le développement limité de  $\cos(x)$ , à l'ordre  $n$ , est :

$$\cos(x) = \sum_{i=0}^n (-1)^i * \frac{x^{2i}}{(2i)!}$$

1 pt Q1- Écrire la fonction **factoriel** ( $k$ ), qui reçoit en paramètre un entier positif  $k$ , et qui retourne la valeur de factorielle  $k$  :  $k! = 1 * 2 * 3 * \dots * (k-1) * k$ .

NB : La fonction **factoriel** (0) retourne 1

1 pt Q2- Écrire la fonction **calcul** ( $x$ ,  $k$ ) qui reçoit en paramètres un réel  $x$  et un entier positif  $k$ , et qui retourne la valeur de l'expression suivante :

$$(-1)^k * \frac{x^{2k}}{(2k)!}$$

0.5 pt Q3- Déterminer la complexité de la fonction **calcul** ( $x$ ,  $k$ ), et justifier votre réponse.

1.5 pt Q4- Écrire la fonction **cosinus** ( $x$ ,  $n$ ) qui reçoit en paramètres un réel  $x$  et un entier positif  $n$ , et qui retourne la valeur du développement limité de  $\cos(x)$ , à l'ordre  $n$ .

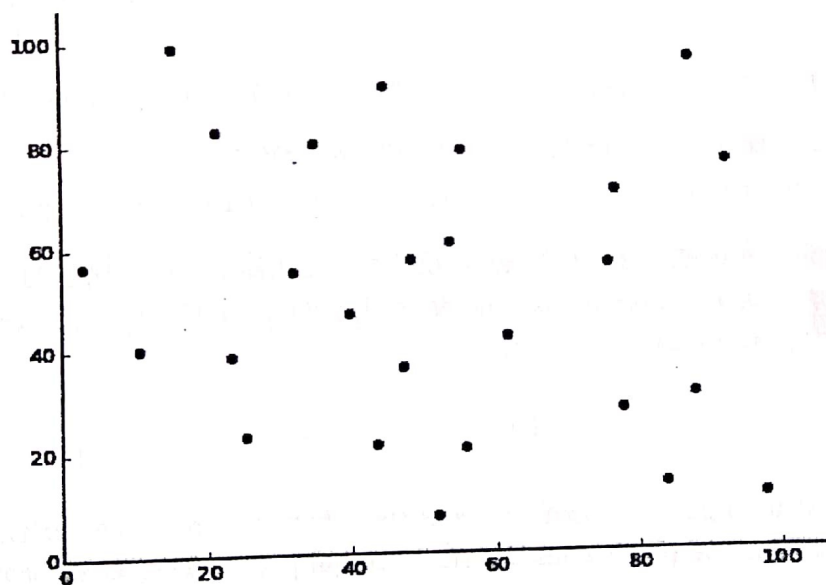
## Partie III : Problème Python

### Géométrie algorithmique

#### *Recherche des deux points les plus rapprochés*

La **géométrie algorithmique** est un domaine qui traite des algorithmes manipulant des concepts géométriques. La discipline qui a le plus contribué historiquement au développement de la géométrie algorithmique est l'infographie. Toutefois, à l'heure actuelle, la géométrie algorithmique se voit fréquemment impliquée dans des problèmes d'algorithmique générale. Aujourd'hui, la géométrie algorithmique est utilisée en infographie, en robotique, pour les systèmes d'information géographique, ...

En géométrie algorithmique, la **recherche des deux points les plus rapprochés** est le problème qui consiste à trouver une paire de points, d'un ensemble fini de points dans un espace métrique, dont la distance est minimale. Il fait partie des problèmes fondateurs de la géométrie algorithmique.



Ce problème de recherche voit son utilité dans les transports aériens ou maritimes par exemple. Il est utilisé par les contrôleurs aériens pour repérer les avions les plus proches les uns des autres (l'espace considéré ici est à 3 dimensions), et ainsi prévenir le risque de collision.

Dans cette partie, nous allons nous intéresser au problème suivant : étant donné un ensemble de points du plan, on cherchera à trouver le couple de points les plus rapprochés au sens de la distance euclidienne.

Un point du plan sera représenté par un tuple de réels  $(x, y)$  représentant ses coordonnées dans un repère orthonormé. L'ensemble des points considérés sera quant à lui stocké dans une liste  $P$ .

Nous supposerons que la liste  $P$  contient au moins deux points, et que les points de  $P$  sont tous différents deux à deux. Nous supposerons aussi que la liste  $P$  contient une seule paire de points, à distance minimale.



**Exemple :**

Les points de la figure précédente sont représentés par la liste **P** suivante :

$P = [ (84.04, 12.68), (21.23, 82.75), (35.00, 80.00), (78.00, 27.00), (52.01, 7.03), (48.59, 56.77), (88.16, 30.04), (77.3, 69.82), (40.00, 46.46), (10.72, 39.91), (97.98, 10.52), (31.86, 54.75), (54.19, 60.09), (87.97, 95.69), (23.21, 38.41), (93.19, 75.47), (55.82, 78.18), (25.21, 22.75), \dots ]$

**NB :** Dans tous les exemples de cette partie, on utilisera cette liste **P**.

**Distance euclidienne entre deux points :**

La distance euclidienne entre deux points **A** et **B** de coordonnées  $(x_A, y_A)$  et  $(x_B, y_B)$ , est calculée par la formule suivante :

$$\sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}$$

**Q.1-** Écrire la fonction *distance* (**A**, **B**) qui reçoit en paramètres deux tuples **A** et **B** qui représentent deux points. La fonction retourne la valeur de la distance euclidienne entre les deux points **A** et **B**.

**Exemple :**

**A** = (84.04, 12.68) et **B** = (21.23, 82.75)

La fonction *distance* (**A**, **B**) retourne le nombre : 94.10048352691925

On suppose que la complexité temporelle de la fonction racine carrée est constante. Dans la suite de cette partie, nous nous intéresserons à la mise au point de deux algorithmes différents, qui permettent de trouver les deux points les plus rapprochés, dans la liste **P**.

**A- Algorithme naïf :**

Le premier algorithme est un algorithme naïf qui consiste à faire une recherche exhaustive : considérer successivement tous les couples de points dans la liste **P**, ce qui peut se faire facilement à l'aide de deux boucles imbriquées.

**Q.2-** Écrire la fonction *plus\_proches* (**P**) qui reçoit en paramètre une liste de points **P**. En utilisant le principe de l'algorithme naïf précédent, la fonction retourne deux tuples qui représentent les deux points les plus rapprochés dans la liste **P**.

**Exemple :**

La fonction *plus\_proche* (**P**) retourne les deux points : (48.59, 56.77) , (54.19, 60.09)

**Q.3-** Déterminer la complexité de la fonction *plus\_proches* (**P**).

Justifier votre réponse.

**B- Algorithme efficace**

Le deuxième algorithme est basé sur le paradigme : *diviser pour régner*. Cet algorithme nécessite de disposer de deux copies de la liste des points  $P$  : l'une triée dans l'ordre croissant des abscisses des points, et l'autre triée dans l'ordre croissant des ordonnées des points.

**Q.4-** Écrire la fonction `tri_points` ( $P, k$ ) qui reçoit en paramètre la liste  $P$ , et  $k$  un entier tel que :  $k \in \{0, 1\}$ . La fonction retourne une copie de la liste  $P$ , triée dans l'ordre croissant du  $k^{\text{ème}}$  élément de chaque tuple de la liste  $P$ .

**Exemples :**

- La fonction `tri_points` ( $P, 0$ ) retourne la liste triée selon les abscisses des points :  
[(2.99, 56.44), (10.72, 39.91), (15.33, 99.03), (21.23, 82.75), (23.21, 38.41), ... ]
- La fonction `tri_points` ( $P, 1$ ) retourne la liste triée selon les ordonnées des points :  
[(52.01, 7.03), (97.98, 10.52), (84.04, 12.68), (56.0, 20.0), (43.7, 20.82), ... ]

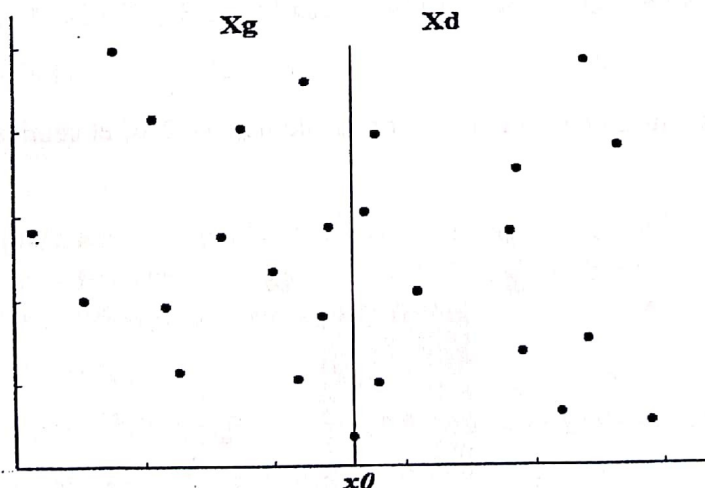
On suppose que la liste  $X$  est la copie de  $P$ , triée dans l'ordre croissant des abscisses des points.

On considère deux variables  $n$  et  $x_0$ , telles que :

- $n \leftarrow$  Longueur de  $X$  ;
- $x_0 \leftarrow$  l'abscisse du point au milieu de la liste  $X$ , à l'indice  $n/2$ .

Ensuite, à partir des éléments de la liste triée  $X$ , on crée deux listes  $X_g$  et  $X_d$ , telles que :

- $X_g$  contient la première moitié de la liste  $X$  : les éléments de  $X$  d'indice  $i$  tel que  $0 \leq i < n/2$  ;
- $X_d$  contient la deuxième moitié de la liste  $X$  : les éléments de  $X$  d'indice  $i$  tel que  $n/2 \leq i < n$ .



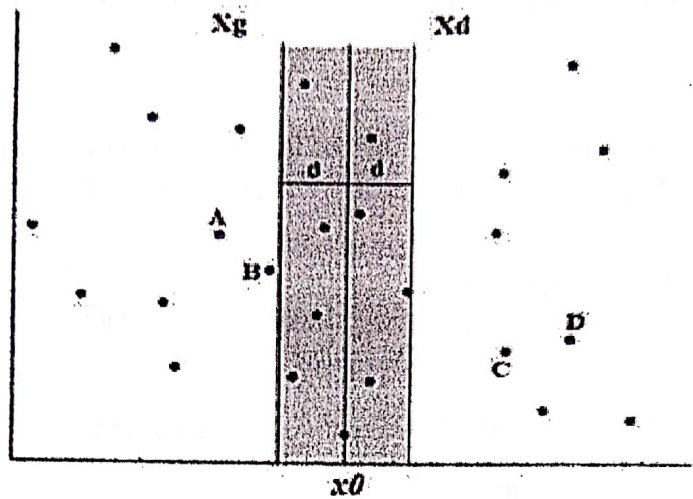
Supposons que  $A$  et  $B$  sont les deux points les plus rapprochés dans la liste  $X_g$ , et que  $d_1$  est la distance entre  $A$  et  $B$ .

Supposons aussi que  $C$  et  $D$  sont les deux points les plus rapprochés dans la liste  $X_d$ , et que  $d_2$  est la distance entre  $C$  et  $D$ .

On pose :  $d \leftarrow$  Le minimum de  $d_1$  et  $d_2$

Si deux points de la liste  $P$  sont à une distance strictement inférieure à  $d$ , alors forcément :

- L'un des deux points appartient à la liste  $X_g$ , et l'autre appartient à la liste  $X_d$  ;
- Et les deux points se trouvent dans la bande verticale qui contient les points d'abscisses comprises entre  $x_0-d$  et  $x_0+d$ .



**Q.5-** Écrire la fonction *bande\_verticale* ( $Y, x_0, d$ ) qui reçoit en paramètres la liste  $Y$  des points triés dans l'ordre croissant des ordonnées, et les deux réels  $x_0$  et  $d$ . La fonction retourne la liste  $B$  des points de  $Y$  dont les abscisses sont comprises entre  $x_0-d$  et  $x_0+d$ , et triés dans l'ordre croissant des ordonnées.

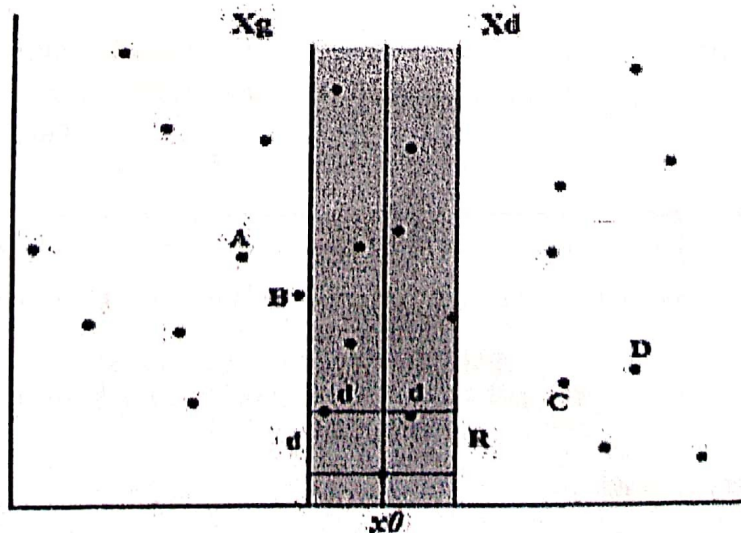
**Exemple :**

$Y = [(52.01, 7.03), (97.98, 10.52), (84.04, 12.68), (56.0, 20.0), (43.7, 20.82), (25.21, 22.75), \dots]$

On suppose que  $x_0 = 52.01$  et  $d = 10.605055398252285$

La fonction *bande\_verticale* ( $Y, x_0, d$ ) retourne la liste  $B$  suivante :  $[(52.01, 7.03), (56.0, 20.0), (43.7, 20.82), (47.54, 35.79), (62.0, 41.5), (48.59, 56.77), (54.19, 60.09), (55.82, 78.18), (45.0, 91.0)]$

La liste  $B$  contient les points situés dans la bande verticale de largeur  $2*d$ , et centrée en la droite verticale passante par l'abscisse  $x_0$ .





Supposons qu'ils existent deux points  $B_i$  et  $B_j$  de la liste  $B$ , tels que la distance entre ces deux points est inférieure à  $d$ . Les points  $B_i$  et  $B_j$  sont donc dans un rectangle  $R$  de dimension  $d \times 2d$  centré en la droite verticale passant par  $x_0$ . On suppose que le point  $B_i$  est sur le côté inférieur du rectangle  $R$ .

Montrons que le rectangle  $R$  contient au plus 8 points de  $B$ . Pour cela, on considère le carré de dimension  $d \times d$  qui forme la moitié gauche du rectangle  $R$  :

- Puisque tous les points de  $X_g$  sont distants d'au moins  $d$ , il y en a au plus 4 points qui se trouvent dans ce carré : 1 point par sous-carré de dimension  $(d/2) \times (d/2)$  de diagonale  $d\sqrt{2}/2 < d$ .
- De même, il y a au plus 4 points qui se trouvent dans la moitié droite de  $R$ .

Donc, le rectangle  $R$  contient au plus 8 points de  $B$ , qui sont classés par ordonnées croissantes, et on a bien :  $i < j \leq i+7$  et la distance entre les points  $B_i$  et  $B_j$  est inférieure à  $d$ .

Ainsi, on déduit que si deux points de  $B$  sont à une distance inférieure à  $d$ , alors ils sont situés à au plus 7 positions l'un de l'autre dans la liste  $B$ .

**Q.7-** Écrire la fonction de complexité linéaire *plus\_proches\_bande* ( $B$ ) qui reçoit en paramètre la liste  $B$  qui contient les points de la bande verticale. La fonction retourne les deux points les plus rapprochés dans la liste  $B$ .

**Exemple :**

$B = [(52.01, 7.03), (56.0, 20.0), (43.7, 20.82), (47.54, 35.79), (62.0, 41.5), (48.59, 56.77), (54.19, 60.09), (55.82, 78.18), (45.0, 91.0)]$

La fonction *plus\_proches\_bande* ( $B$ ) retourne les deux points :  $(48.59, 56.77)$  ,  $(54.19, 60.09)$

**Q.8-** Justifier que la fonction *plus\_proches\_bande* ( $B$ ) est de complexité linéaire.

En utilisant les fonctions précédentes, on peut mettre en point un algorithme récursif permettant de trouver les deux points les plus rapprochés à partir des deux listes  $X$  (liste des points triés par abscisses croissantes) et  $Y$  (liste des points triés par ordonnées croissantes) :

- $n \leftarrow$  longueur de la liste  $X$
- Si  $n \leq 3$  alors retourner les deux points les plus rapprochés, en utilisant l'algorithme naïf ;
- $x_0 \leftarrow$  l'abscisse du point au milieu de la liste  $X$
- Créer deux listes  $X_g$  et  $X_d$ , qui contiennent respectivement la première moitié et la deuxième moitié de la liste  $X$  ;
- Avec un appel récursif, déterminer les deux points  $A$  et  $B$  les plus rapprochés dans  $X_g$  ;
- $d_1 \leftarrow$  la distance entre  $A$  et  $B$  ;
- Avec un appel récursif, déterminer les deux points  $C$  et  $D$  les plus rapprochés dans  $X_d$  ;

- h.  $d_2 \leftarrow$  la distance entre C et D ;
- i. Déterminer la paire de points (E, F) les plus rapprochés parmi les paires (A, B) et (C, D), ainsi que la distance d entre E et F ;
- j. Déterminer la liste B qui représente la bande verticale de largeur  $2*d$ , centrée en la droite verticale passant par  $x_0$  ;
- k. Si la longueur de la liste B est inférieure strictement à 2, alors on retourne la paire de points (E, F) ;
- l. Si la liste B contient au moins deux points, alors déterminer les deux points P et Q les plus rapprochés dans B, ainsi que la distance  $d_3$  entre P et Q ;
- m. Retourner la paire de points à distance minimale parmi les paires (E, F) et (P, Q)

Q.9- Écrire la fonction récursive *plus\_proches\_rec* (X, Y) qui reçoit en paramètres les deux listes X et Y, et qui retourne les deux points les plus rapprochés.

Exemple :

X = [(2.99, 56.44), (10.72, 39.91), (15.33, 99.03), (21.23, 82.75), (23.21, 38.41), ... ]

Y = [(52.01, 7.03), (97.98, 10.52), (84.04, 12.68), (56.0, 20.0), (43.7, 20.82), ... ]

La fonction *plus\_proches\_rec* (X, Y) retourne les points : (48.59, 56.77) , (54.19, 60.09)

Q.10- Déterminer la complexité de la fonction *plus\_proches\_rec* (X, Y), et justifier votre réponse.

===== FIN DE L'ÉPREUVE =====